数据库管理系统原理与实现

杜小勇 陈红 卢卫 主编

深圳技术大学

大数据与互联网学院

教学目标

- 内容概述：SQL是关系数据库的标准语言。讲解SQL的同时要进一步讲述关系数据库的基本概念。
- 本章目标：牢固掌握SQL，达到举一反三的掌握SQL的功能。同时通过实践，体会面向过程的语言和SQL的区别和优点。体会关系数据库系统为数据库应用系统的开发提供良好环境，减轻用户负担，提高用户生产率的原因。
- 重点：第3、4章以及数据库设计是本学期重点。要熟练正确的使用SQL完成对数据库的查询、插入、删除、更新操作。
- 难点：用SQL语言正确完成复杂查询，掌握SQL语言强大的查询功能。因此一定要多编程，要在安装好的数据库系统上进行实际操作，检查你的答案，你查询的结果是否正确。只有通过大量练习才能真正达到举一反三的熟练程度。

第3章 关系数据库标准语言SQL

- 3.1 SQL概述
- 3.2 数据定义
- 3.3 基本SQL查询
- 3.4 数据更新
- 3.5 视图
- 3.6 小结

3.1 SQL概述

SQL（structured query language）是一种非过程化(不具体描述程序处理过程即可获取所需的输出)语言。

SQL集数据定义、数据查询、数据操纵和数据控制等功能于一体，具有功能综合、风格统一、数据操纵高度非过程化、面向集合操作、语法结构统一和语言简洁等特点。

3.1 SQL的特点

- 综合统一
  - 集数据定义语言（DDL），数据操纵语言（DML），数据控制语言（DCL）功能于一体。
  - 可以独立完成数据库生命周期中的全部活动：
    - 定义和修改、删除关系模式，定义和删除视图，插入数据，建立数据库；
    - 对数据库中的数据进行查询和更新；
    - 数据库重构和维护
    - 数据库安全性、完整性控制，以及事务控制
    - SQL编程
  - 用户数据库投入运行后，可根据需要随时逐步修改模式，不影响数据库的运行。
  - 数据操作符统一

2. 高度非过程化

- 非关系数据模型的数据操纵语言“面向过程”，必须指定存取路径。
- SQL只要提出“做什么”，无须了解存取路径。
- 存取路径的选择以及SQL的操作过程由系统自动完成。

3. 面向集合的操作方式(通过元组)

- 非关系数据模型采用面向记录的操作方式，操作对象是一条记录
- SQL可以操作集合(本质依然是通过元组)
  - 操作对象、查找结果可以是元组的集合
  - 一次插入、删除、更新操作的对象可以是元组的集合

4.语言简洁，门槛低，但精通复杂应用难

- SQL功能极强，但核心功能只用了9个动词。

表 3.2　SQL 的动词

SQL 功　能	动词
数 据 查 询	SELECT
数 据 定 义	CREATE，DROP，ALTER
数 据 操 纵	INSERT，UPDATE，DELETE
数 据 控 制	GRANT，REVOKE

SQL标准的进展过程

标准	大致页数	发布日期
SQL/86		1986.10
SQL/89（FIPS 127-1）	120页	1989年
SQL/92	622页	1992年
SQL99（SQL 3）	1700页	1999年
SQL2003	3600页	2003年
SQL2008	3777页	2006年
SQL2011		2010年

目前，没有一个数据库系统能够支持SQL标准的所有概念和特性。

教材以SQL92为基础(可兼顾oracle/sql server)，ppt会加入mysql5.7/8.0对应有区别的语法(OJ实验和考试按Mysql语法)

第3章 关系数据库标准语言SQL

- 3.1 SQL概述
- 3.2 数据定义
- 3.3 基本SQL查询
- 3.4 数据更新
- 3.5 视图
- 3.6 小结

3.2 数据定义

- 3.2.1 模式的定义与删除
- 3.2.2 基本表的定义、修改与删除
- 3.2.3 索引的建立、修改与删除

❖SQL的数据定义功能:

- 模式定义
- 表定义
- 视图和索引的定义

表 3.3　SQL 的数据定义语句

操作对象	操作方式		
	创　建	删　除	修　改
模式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视图	CREATE VIEW	DROP VIEW	
索引	CREATE INDEX	DROP INDEX	ALTER INDEX

1. 模式（Schema）

- **模式（也称逻辑模式）**
  - 数据库中全体数据的逻辑结构和特征的描述
  - 所有用户的公共数据视图
  - 独立于数据库的其他层次，设计数据库模式结构时应首先确定数据库的逻辑模式
- **模式的地位：是数据库系统模式结构的中间层**
  - 与数据的物理存储细节和硬件环境无关
  - 与具体的应用程序、开发工具及高级程序设计语言无关

模式（续）

- **模式的定义**
  - 数据的逻辑结构（数据项的名字、类型、取值范围等）
  - 数据之间的联系
  - 数据有关的安全性、完整性要求

❖一个数据库只有一个模式

Mysql：从概念上讲，模式是一组相互关联的数据库对象，例如表，表列，列的数据类型，索引，外键等。
在MySQL中，模式与数据库是同义词。您可以用MySQL SQL语法DATABASE 代替SCHEMA关键字，例如使用**CREATE DATABASE** 代替CREATE SCHEMA 。

定义模式

- 定义模式实际上定义了一个命名空间。
- 在这个空间中可以定义该模式包含的数据库对象，例如基本表、视图、索引等。
- 在CREATE SCHEMA中可以接受CREATE TABLE, CREATE VIEW和GRANT子句。

CREATE SCHEMA <模式名> AUTHORIZATION <用户名>[<表定义子句>|<视图定义子句>|<授权定义子句>]

命令行语法：

[] 表示里面的元素（参数、值或信息）是可选的

尖括号 < > 表示里面的元素(参数、值或信息）是必需的

3.2.1 模式的定义与删除

想要创建Online-Shopping模式，可使用如下语句：

CREATE SCHEMA “Online-Shopping”

本章被方框包含的内容皆表示语句的一般格式。

语句的语法格式如下：

```
<schema definition> ::=
CREATE SCHEMA <schema name clause>
[ <schema character set or path> ] [ <schema element>... ]
```

- 符号“::=”表示对一个元素的定义；符号“|”后面接的是可选内容。
- 不带括号的字符串如“CREATE SCHEMA”表示必须保留的关键字。
- 带尖括号的字符串如“<schema name clause>”表示一个元素(SCHEMA)。

3.2.1 模式的定义与删除

定义模式名称子句时可以指定模式名称或被授权的用户，也可以两者同时指定。

<schema name clause> ::=

<schema name>

| AUTHORIZATION <schema authorization identifier>

| <schema name> AUTHORIZATION <schema authorization identifier>

<schema authorization identifier> ::= <authorization identifier>

[例3.1]为用户张三创建一个模式Online-Shopping。

CREATE SCHEMA “Online-Shopping” AUTHORIZATION “张三”；

Mysql定义模式

CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] *db_name* <数据库名>
[*create_option*] ... *create_option*: [DEFAULT] { CHARACTER SET [=] *charset_name* <字符集名>
| COLLATE [=] *collation_name* <校对规则名>
| ENCRYPTION [=] {'Y' | 'N'} } //加密

- IF NOT EXISTS：在创建数据库之前进行判断，只有该数据库目前尚不存在时才能执行操作。此选项可以用来避免数据库已经存在而重复创建的错误。
- [DEFAULT] CHARACTER SET：指定数据库的字符集。指定字符集的目的是为了避免在数据库中存储的数据出现乱码的情况。如果在创建数据库时不指定字符集(utf8)，那么就使用系统的默认字符集。
- [DEFAULT] COLLATE：指定字符集的默认校对规则(utf8_chinese_ci)。

Mysql修改模式（续）

```
ALTER DATABASE <数据库名>
[[DEFAULT] CHARACTER SET <字符集名>]
[[DEFAULT] COLLATE <校对规则名>];
```

用于更改数据库的全局特性

数据库命名

在创建数据库时，MySQL数据库命名有以下几项规则：

- 不能与其他数据库重名，否则将发生错误。
- 名称可以由任意字母、阿拉伯数字、下划线（_）和“$”组成，可以使用上述的任意字符开头，但不能使用单独的数字，否则会造成它与数值相混淆。
- 名称最长可为64个字符，而别名最多可长达256个字符。
- 不能使用MySQL关键字作为数据库名、表名。
- 在默认情况下，Windows下数据库名、表名的大小写是不敏感的(因为文件系统如此)，而在Linux下数据库名、表名的大小写是敏感的。为了便于数据库在平台间进行移植，建议采用小写来定义数据库名和表名。

数据库命名

编程规范：

【强制】表名、字段名必须使用小写字母或数字，禁止出现数字开头，禁止两个下划线中间只出现数字。数据库字段名的修改代价很大，因为无法进行预发布，所以字段名称需要慎重考虑。

字段名不推荐使用mysql关键字，否则需要用esc键下方的``转义符以区别关键字，否则报错。

SQL语句主要规则

SQL 语句以分号;结尾

SQL 语句中的标点符号包括空格必须都是英文状态下，即半角字！(因为装了中文输入法，容易错！)

SQL 默认不区分关键字/列名的大小写。(均可以设置)

例如，不管写成 SeLECT 还是 sElect，均可。

不区分大小写：存储函数、存储过程和事件的名字。区分大小写：触发器的名字。

推荐，一般统一用小写。

单行注释可以使用#注释符 或 -- 注: --后是空格

多行注释使用/* */注释符

数据内容大小写问题

1、 设置字符集:
utf8_general_ci --不区分大小写(常用)
utf8_bin 或 _cs--区分大小写
•ci 为 case insensitive 默认大小写不敏感
•cs case sensitive 大小写敏感

2、MySql默认查询是不区分大小写的,如果需要区分,必须在建表的时候,Binary(判断二进制/ascii码)标示敏感的属性.

```
CREATE TABLE NAME(
name VARCHAR(10) BINARY
);
```

3、 在SQL语句中实现 SELECT * FROM TABLE NAME WHERE BINARY name='Clip';

3.2.1 模式的定义与删除

[例3.2]删除例3.1中创建的模式。

DROP SCHEMA “Online-Shopping” CASCADE;

<drop schema statement> ::=

DROP SCHEMA <schema name> <drop behavior>

<drop behavior> ::= CASCADE | RESTRICT

在模式的删除行为中，CASCADE和RESTRICT两者必选其一。

- CASCADE（级联）表示同时把该模式中所有的数据库对象全部删除。
- RESTRICT（限制）表示如果该模式中已经定义了数据库对象（如表、 视图等），则拒绝该删除语句的执行；只有当该模式中没有任何数据库对象时才能执行DROP SCHEMA语句。

MySQL删除数据库

DROP DATABASE [IF EXISTS] <数据库名>

MySQL查看数据库

SHOW DATABASES [**LIKE** '数据库名'];

系统生成的库：

1. information_schema：主要存储了系统中的一些数据库对象信息，比如用户表信息、列信息、权限信息、字符集信息和分区信息等。
2. mysql：MySQL 的核心数据库，主要负责存储数据库用户、用户访问权限等 MySQL 自己需要使用的控制和管理信息。常用的比如在 mysql 数据库的 user 表中修改 root 用户密码。
3. performance_schema：主要用于收集数据库服务器性能参数。
4. sys：sys 数据库主要提供了一些函数、存储过程、视图，数据都来自于 performation_schema，主要是让开发者和使用者更方便地查看性能问题。

MySQL选择数据库

在 MySQL 中就有很多系统自带的数据库，那么在操作数据库之前就必须要确定是哪一个数据库。

USE <数据库名>

3.2 数据定义

- 3.2.1 模式的定义与删除
- 3.2.2 基本表的定义、修改与删除
- 3.2.3 索引的建立、修改与删除

数据类型

- SQL中域的概念用数据类型来实现
- 定义表的属性时需要指明其数据类型及长度
- 选用哪种数据类型
  - 取值范围
  - 要做哪些运算

MySQL数值类型

类型	大小	范围（有符号）	范围（无符号）	用途
TINYINT	1 Bytes	(-128，127)	(0，255)	小整数值
SMALLINT	2 Bytes	(-32 768，32 767)	(0，65 535)	大整数值
MEDIUMINT	3 Bytes	(-8 388 608，8 388 607)	(0，16 777 215)	大整数值
INT或INTEGER	4 Bytes	(-2 147 483 648，2 147 483 647)	(0，4 294 967 295)	9位整数安全
BIGINT	8 Bytes	(-9,223,372,036,854,775,808，9 223 372 036 854 775 807)	(0，18 446 744 073 709 551 615)	19位整数安全
FLOAT	4 Bytes	(-3.402 823 466 E+38，-1.175 494 351 E-38)，0，(1.175 494 351 E-38，3.402 823 466 351 E+38)	0，(1.175 494 351 E-38，3.402 823 466 E+38)	单精度 浮点数值
DOUBLE	8 Bytes	(-1.797 693 134 862 315 7 E+308，-2.225 073 858 507 201 4 E-308)，0，(2.225 073 858 507 201 4 E-308，1.797 693 134 862 315 7 E+308)	0，(2.225 073 858 507 201 4 E-308，1.797 693 134 862 315 7 E+308)	双精度 浮点数值
DECIMAL	对DECIMAL(M,D)，如果M>D，为M+2否则为D+2	依赖于M和D的值	依赖于M和D的值	小数值

MySQL字符串类型

类型	大小	用途
CHAR	0-255 bytes	定长字符串
VARCHAR	0-65535 bytes	变长字符串
TINYBLOB	0-255 bytes	不超过 255 个字符的二进制字符串
TINYTEXT	0-255 bytes	短文本字符串
BLOB	0-65 535 bytes	二进制形式的长文本数据
TEXT	0-65 535 bytes	长文本数据
MEDIUMBLOB	0-16 777 215 bytes	二进制形式的中等长度文本数据
MEDIUMTEXT	0-16 777 215 bytes	中等长度文本数据
LONGBLOB	0-4 294 967 295 bytes	二进制形式的极大文本数据
LONGTEXT	0-4 294 967 295 bytes	极大文本数据

MySQL日期和时间类型

类型	大小 (bytes)	范围	格式	用途
DATE	3	1000-01-01/9999-12-31	YYYY-MM-DD	日期值
TIME	3	'-838:59:59'/'838:59:59'	HH:MM:SS	时间值或持续时间
YEAR	1	1901/2155	YYYY	年份值
DATETIME	8	1000-01-01 00:00:00/9999-12-31 23:59:59	YYYY-MM-DD HH:MM:SS	混合日期和时间值
TIMESTAMP	4	1970-01-01 00:00:00/2038结束时间是第 **2147483647** 秒，北京时间 **2038-1-19 11:14:07**，格林尼治时间 2038年1月19日凌晨 03:14:07	YYYYMMDD HHMMSS	混合日期和时间值，时间戳

“网上商城”数据库示例(期中学习数据库设计后再回顾表设计):

Customers

顾客ID (CID)	顾客姓名 (CName)	城市 (City)
1001	张三	北京
1002	李四	广州
1003	王五	上海

Suppliers

供应商ID (SID)	供应商名称 (SName)	城市 (City)
S001	华北手机厂	北京
S002	西北电子厂	西安
S003	未央印刷厂	长安

Orders

订单编号 (OID)	顾客ID (CID)	创建时间 (CreateTime)
O001	1001	2023-1-1 18:00:10
O002	1002	2023-1-1 18:20:01
O003	1002	2023-1-5 09:10:20

Products

商品ID (PID)	商品名称 (PName)	价格 (Price)	种类 (Category)	供应商ID (SID)
P0001	智能手机	1999	数码产品	S001
P0002	老人专用手机	899	数码产品	S001
P0003	平板电脑	1688	数码产品	S002
P0004	数据库教材	48	书籍	S003
P0005	流浪火星	65	书籍	S003

OrderItems

订单编号 (OID)	商品ID (PID)	数量（Quantity）	折扣（Discount）
O001	P0001	1	0.85
O001	P0004	20	0.9
O001	P0005	10	0.9
O002	P0002	1	0.9
O003	P0003	1	0.7
O004	P0001	100	0.8

3.3.2 基本表的定义、删除与修改（第6版）

- 定义基本表

  CREATE TABLE <表名>
  (<列名> <数据类型>[<列级完整性约束条件>]
  [,<列名> <数据类型>[<列级完整性约束条件>]]
  ...
  [,<表级完整性约束条件>]);

  - <表名>：所要定义的基本表的名字
  - <列名>：组成该表的各个属性（列）
  - <列级完整性约束条件>：涉及相应属性列的完整性约束条件
  - <表级完整性约束条件>：涉及一个或多个属性列的完整性约束条件
  - 如果完整性约束条件涉及到该表的多个属性列，则必须定义在表级上，否则既可以定义在列级也可以定义在表级。

3.2.2 基本表的定义、修改与删除

若要创建顾客表可使用如下语句：

顾客ID (CID)	顾客姓名 (CName)	城市 (City)

```
CREATE TABLE Customers(
        CID VARCHAR(32) PRIMARY KEY,
        CName VARCHAR(128) NOT NULL,
        City VARCHAR(128)
);
```

<table definition> ::=

CREATE [{ GLOBAL | LOCAL } TEMPORARY] TABLE <table name>

<table element list> [ON COMMIT { DELETE | PRESERVE } ROWS]

<table element list>指表的结构的部分，包括列定义、数据类型和约束等信息。

3.2.2 基本表的定义、修改与删除

```
CREATE TABLE Customers(
        CID VARCHAR(32) PRIMARY KEY,
        CName VARCHAR(128) NOT NULL,
        City VARCHAR(128)
);
```

<table element list> ::= <left paren> <table element> [{ <comma> <table element> }...] <right paren>
<table element> ::= <column definition> | <table constraint definition>

3.2.2 基本表的定义、修改与删除

```
CREATE TABLE Customers(
        CID VARCHAR(32) PRIMARY KEY,
        CName VARCHAR(128) NOT NULL,
        City VARCHAR(128)););
```

```
<column definition> ::=
<column name> { <data type> | <domain name> }
[ <default clause> ] [ <column constraint definition>... ] [ <collate clause> ]
```

<column constraint definition>**列级完整性约束**，涉及相应属性列的完整性约束，下一页详解。

3.2.2 基本表的定义、修改与删除

<column constraint definition>至少要包含<column constraint>。

< column constraint definition> ::=

[<constraint name definition>] <column constraint> [<constraint attributes>]

[<constraint name definition>]指定约束的名称。
<column constraint>是具体的列约束类型，主键、外键等。
[<constraint attributes>]指定额外的约束属性。

<column constraint>
可以是not null（不允许空值）、
unique （唯一性约束）、
foreign key references（外键约束）、或者
check （check约束）。

3.2.2 基本表的定义、修改与删除

```
CREATE TABLE Products(
        PID VARCHAR(32) PRIMARY KEY,        /*商品ID （主码） */
        PName VARCHAR(128) NOT NULL,        /*商品名称*/
        SID VARCHAR(32) NOT NULL,           /*供应商ID*/
        FOREIGN KEY (SID) REFERENCES Suppliers(SID)
);
```

<table element list> ::=

<left paren> <table element> [{ <comma> <table element> }...] <right paren>

<table element> ::= <column definition> | <table constraint definition>

3.2.2 基本表的定义、修改与删除

<table constraint definition>表级完整性约束，涉及一个或多个属性列的完整性约束。

<table constraint definition> ::=

[<constraint name definition>] <table constraint> [<constraint attributes>]

<table constraint> ::=

PRIMARY KEY

| <unique >

| <foreign key references>

| <check >

3.2.2 基本表的定义、修改与删除

顾客ID (CID)	顾客姓名 (CName)	城市 (City)

[例3.3]创建顾客表。

```
CREATE TABLE Customers(
        CID VARCHAR(32) PRIMARY KEY,   /*顾客ID*/
        CName VARCHAR(128) NOT NULL,   /*顾客姓名*/
        City VARCHAR(128)                    /*顾客所在城市*/
);
```

VARCHAR表示其为可变长度的字符串，最大长度为32。 PRIMARY KEY是列级约束条件，表示这是主码。

NOT NULL也是列级约束条件，表明其不可为空值。

3.2.2 基本表的定义、修改与删除

供应商ID (SID)	供应商名称 (SName)	城市 (City)

[例3.4]创建供应商表。

```
CREATE TABLE Suppliers(
        SID VARCHAR(32) PRIMARY KEY,    /*供应商ID（主码）*/
        SName VARCHAR(128) NOT NULL,    /*供应商名称*/
        City VARCHAR(128)                /*供应商所在城市*/
);
```

3.2.2 基本表的定义、修改与删除

商品ID (PID)	商品名称 (PName)	价格 (Price)	种类 (Category)	供应商ID (SID)

[例3.5]创建商品表。

```
CREATE TABLE Products(
        PID VARCHAR(32) PRIMARY KEY,        /*商品ID （主码） */
        PName VARCHAR(128) NOT NULL,        /*商品名称*/
        Price DECIMAL,                      /*商品单价，类型为带小数的数字*/
        Category VARCHAR(128),              /*商品类别*/
        SID VARCHAR(32) NOT NULL,           /*供应商ID*/
        FOREIGN KEY (SID) REFERENCES Suppliers(SID)
);
```

定义了一个外码约束，表示SID属性是一个外码，引用了Suppliers表的SID属性。

3.2.2 基本表的定义、修改与删除

[例3.6]创建订单表。

订单编号 (OID)	顾客ID (CID)	创建时间 (CreateTime)

```
CREATE TABLE Orders(
        OID VARCHAR(32) PRIMARY KEY,  /*订单ID（主码）*/
        CID VARCHAR(32),                          /*顾客ID*/
        CreateTimeName DATETIME
         /*订单创建时间，DATETIME是时间数据类型 */
        FOREIGN KEY (CID) REFERENCES Customers(CID)
);
```

定义了一个外码约束，表示CID属性是一个外键，引用了Customers表中的CID属性。

3.2.2 基本表的定义、修改与删除

[例3.7]创建订单项表。

订单编号(OID)	商品ID(PID)	数量（Quantity）	折扣（Discount）

```
CREATE TABLE OrderItems(
        OID VARCHAR(32),        /*订单ID*/
        PID VARCHAR(32),        /*商品ID*/
        Quantity INT,           /*数量*/
        Discount DECIMAL,       /*折扣*/
        PRIMARY KEY(OID, PID), /*多列，只能用表级完整性*/
        FOREIGN KEY (OID) REFERENCES Orders(OID),
        FOREIGN KEY (PID) REFERENCES Products(PID)
);
```

表的主码由OID和PID共同组成。OID和PID都是外码，分别引用了订单表的OID和商品表的PID。

Mysql删除基本表

DROP [TEMPORARY] TABLE [IF EXISTS]

tbl_name [, *tbl_name*] ...

[RESTRICT | CASCADE] #mysql 8.0实际也不生效，图形界面选择是否外键约束。

Mysql 外码约束

RESTRICT|NO ACTION|SET NULL|CASCADE

1. RESTRICT：当基表删除|更新记录时，首先检查记录是否有对应的外码，如有，报错。
2. NO ACTION 同RESTRICT。
3. SET NULL：如有，则设置参照表对应的外键值为null（前提外键非主属性，允许为null）。
4. CASCADE：如有，则设置子表对应的外码值跟随删除及更新对应记录。

修改基本表

ALTER TABLE <表名>

[ADD[COLUMN] <新列名> <数据类型> [完整性约束]]

[ADD <表级完整性约束>]

[DROP [COLUMN] <列名> [CASCADE| RESTRICT]]

[DROP CONSTRAINT<完整性约束名>[RESTRICT | CASCADE]]

[ALTER COLUMN <列名><数据类型>] ;

https://dev.mysql.com/doc/refman/8.0/en/alter-table.html

修改基本表（续）

■<表名>是要修改的基本表

■ADD子句用于增加新列、新的列级完整性约束条件和新的表级完整性约束条件

■DROP COLUMN子句用于删除表中的列

- 如果指定了CASCADE短语，则自动删除引用了该列的其他对象
- 如果指定了RESTRICT短语，则如果该列被其他对象引用，关系数据库管理系统将拒绝删除该列

■DROP CONSTRAINT子句用于删除指定的完整性约束条件

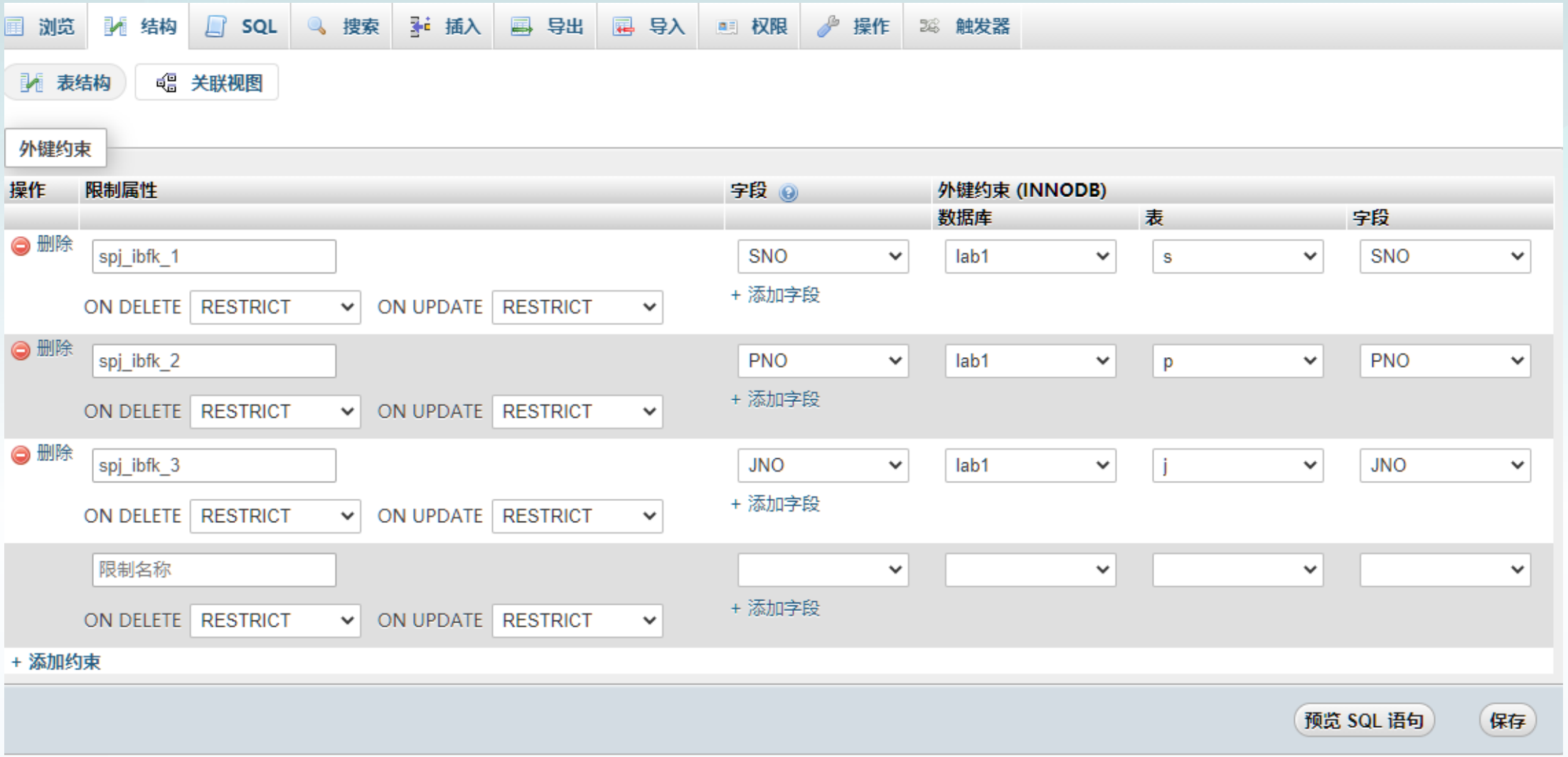
■ALTER COLUMN子句用于修改原有的列定义，包括修改列名和数据类型

Mysql 语法

https://dev.mysql.com/doc/refman/8.0/en/create-table.html

1. auto_increment参数指定字段为自增字段，该字段的类型必须为int或bigint才能设置为自增字段。
2. not null|null参数设置字段能否取空值。
3. default参数设置字段的默认值。
4. index参数用来创建索引。idx_name指定索引名，该参数可以省略，如果省略则索引名就是字段名。
5. foreign key参数用来指定外键。
6. engine参数用来设置引擎类型，常用的有innoDB和myISAM引擎。
7. default charset参数设置表所用的字符集。
8. 。。。

第一次实验课：常使用开发工具界面建库建表



3.2 数据定义

- 3.2.1 模式的定义与删除
- 3.2.2 基本表的定义、修改与删除
- 3.2.3 索引的建立、修改与删除

3.2.3 索引的建立与删除

- 建立索引的目的：加快查询存储的速度(第6章详解)
- 经常需要连接(比如外码)或查询或排序或分组的字段

最合适的类比是新华字典(多种目录对应多个索引，聚簇索引就是拼音目录)

索 引

- 谁可以建立索引
  - 数据库管理员 或 表的属主（拥有权限）
- 谁维护索引
  - 关系数据库管理系统自动完成
- 使用索引
  - 关系数据库管理系统自动选择合适的索引作为存取路径，用户不必也不能显式地选择索引

3.2.3 索引的建立、修改与删除

[例3.8]在顾客表的姓名列上建立索引。

CREATE INDEX idx-cname ON Custermers (CName);

建立索引语句的语法格式如下：

CREATE [CLUSTERED] INDEX name ON table_name ({column_name|(expression)}

3.2.3 索引的建立、修改与删除

创建索引需要为索引指定唯一的名称，并指定表的名称和属性列的名称。

```
CREATE [CLUSTERED] INDEX name ON table_name ({column_name|(expression)}
```

CLUSTERED是可选的关键字，用于指示要创建的索引是否是聚集索引(Mysql不按此语法)。

聚集索引要求数据库管理系统根据指定属性列的值来存储元组。

删除索引

- DROP INDEX <索引名>;

  删除索引时，系统会从数据字典中删去有关该索引的描述。

Mysql建立索引命令行

因为表已建好，聚簇索引就是主码(不指定主码，会隐含一个聚簇索引，商用时必须指定)。

未指定主码时：

```
ALTER TABLE tableName ADD PRIMARY KEY
(columnName,…);  #允许多列
```

非聚簇索引语句格式：

```
ALTER TABLE tableName ADD INDEX|UNIQUE ind
exName(columnName,…);  #允许多列
```